# SkillLoop: A Reciprocal Skill-Exchange Platform with Rule-Based Match Scoring, Token Incentives and Live Collaboration Rooms

Veerajebika
AIML
Vels university
Chennai, India
veerajebika@gmail.com

Pooja S.
AIML
Vels university
Chennai, India
Poojasenthil052007@gmail.com

Gracy Rose. A
AIML
Vels university
Chennai, India
zeogracyrose@gmail.com

*Project Guide: Dr. G. Revathy*

AIML Vels university, Chennai ,India

*Ai*

***Abstract*—SkillLoop is a web-based platform for peer-to-peer skill exchange. Users list the skills they can teach and the skills they want to learn, and the platform pairs them with members whose lists complement their own. The methodology is modular and covers registration and authentication, skill and profile input, reciprocal matching, recommendation with textual reasons, exchange requests, and collaboration through live rooms, mentor bookings and a token wallet. Matching compares lower-cased skill names in both directions and assigns a rule-based score from 1 to 100, in which every reciprocal pair ranks above every one-way pair. A second ranking adds an activity term and generates a reason for each suggestion. Tokens are spent on rooms and mentor sessions and earned by teaching, within anti-abuse limits. The platform is built in Python with Django and SQLite, and includes a self-hosted captcha, a honeypot field, login throttling, text chat, browser-based video calls and a rule-based assistant. It also has clear limitations. Skills are matched as exact strings, the score constants have not been calibrated on user data, the project-recommendation section proposed in the base work is not implemented, no learned model is used for matching or recommendation, and no user study or load test has been carried out. The evidence reported is therefore a verified module inventory, worked scoring examples and an automated test suite.**

***Keywords*—Skill Exchange, Peer Learning, Reciprocal Matching, Recommender System, Learning by Teaching, Django, WebRTC***

## I. Introduction

Technical skills such as programming, data analysis and design are often learned outside the classroom, where many learners have no easy access to an instructor or a paid mentor. One response is peer learning, in which people who are not professional teachers help one another learn [2], and a skill-exchange platform built on that idea has recently been proposed [1]. Work on collaborative learning describes how interaction between learners can set off activities such as explanation, questioning and the resolution of disagreement [3]. The partner who explains also gains: studies of peer tutoring look at how tutors build knowledge from the explanations and questions they produce [4], and in the experiments of Fiorella and Mayer, participants who actually taught a lesson did better on a test one week later than participants who had only prepared to teach it [5]. These findings motivate platforms in which each member is both a learner and a teacher.

Getting such exchanges to happen is a matching problem. A learner who wants to study Python is best served by someone who knows it, but that person stays engaged only if the exchange offers something in return. A reciprocal condition, in which each side can teach what the other wants to learn, makes the arrangement self-sustaining. Course catalogues and general discussion forums do not model this two-sided condition, so finding a partner is left to chance. The base paper [1] describes a platform with an intelligent matching agent and a project-recommendation section, but it is conceptual and reports neither an implementation nor an evaluation. It therefore leaves open how a reciprocal matching rule would be specified, how matched users would actually meet, and how the system would resist abuse.

This paper presents **SkillLoop**, an implemented skill-exchange platform that brings three parts together in one browser-based system: *reciprocal matching* of learners from self-declared skill lists, *explainable recommendation* of partners, and *collaboration support* through requests, live rooms, mentors and a token wallet. The main contributions are:

- A reciprocal matching procedure with an interpretable 1–100 score, specified formally from the source code, in which reciprocal pairs always rank above one-way pairs.
- A recommendation layer that adjusts the match score with an activity term and generates a short reason for each suggestion.
- A token mechanism with an append-only ledger and anti-abuse limits, which links access to rooms and mentor sessions to teaching.
- Live rooms with text chat and browser-based video calls, together with a self-hosted captcha, a honeypot field and login throttling, all within a single Django application.
- An explicit mapping between what the base work [1] proposes and the features that actually exist in the code.

## II. Related Work

Topping describes peer learning as learning in which people from similar social groups, none of them professional teachers, support one another [2], and Dillenbourg treats collaborative learning as situations in which particular forms of interaction trigger learning mechanisms [3]. Roscoe and Chi analyse how tutors learn from the explanations and

questions that arise in peer tutoring [4], and Fiorella and Mayer separated the effect of preparing to teach from that of actually teaching [5]. These classroom and laboratory studies support the motivation for reciprocal exchange but do not show that any particular platform produces such gains.

Recommender systems are usually divided into content-based, collaborative and hybrid approaches [6], [7]. Manouselis et al. survey recommenders in technology-enhanced learning, most of which suggest items such as courses or resources to a learner [8]. Recommending one person to another is a different task, since the outcome depends on both parties. Pizzato et al. introduced reciprocal recommenders for settings in which a suggestion works only if both people are satisfied, and built RECON for online dating, which estimates compatibility in each direction and combines the two into one score [9]; Palomares et al. review this area and list mentoring among its application domains [10]. Problem-based and project-based learning [11], [12] underlie the project recommendation proposed in the base work [1].

### A. Research Gap

In a typical course platform the roles are fixed: instructors supply content and learners consume it, and nothing represents what a learner could offer in return. The base work [1] proposes a seven-step workflow (registration, skill input, profile storage, skill comparison, match identification, project recommendation and collaboration) and suggests vector embeddings, trade cycles, a reputation system, notifications, gamification and asynchronous processing. However, it does not specify the matching rule precisely enough to be inspected, does not describe how a matched pair would actually meet, and does not discuss abuse. SkillLoop addresses these three points within the scope of a student prototype, as Table I shows. It does not address embeddings, reputation ranking or project recommendation.

TABLE I
BASE-PAPER PROPOSALS VERSUS THE SKILLLOOP CODE

| Proposal in base work [1] | Status in SkillLoop source code |
|---|---|
| Profiles with skills held and wanted | Implemented (UserProfile and Skill models) |
| Complementary skill matching | Implemented: set intersection in both directions, score 1–100 |
| Vector-embedding proximity | Not implemented; names compared as lower-cased strings |
| Bidirectional “trade cycles” | Pairwise reciprocity only; no cycles of three or more |
| Reputation / verification ranking | Not used for ranking; learner confirmation gates token rewards |
| Project recommendation section | Not implemented; no model, view or template exists |
| Collaboration between matched users | Implemented: requests, live rooms, mentor booking |
| Match notifications | Not implemented; pending requests shown on dashboard |
| Gamification | Partial: token wallet and ledger; no badges or leaderboards |
| Secure coordination | Partial: login, CSRF, captcha; no private messaging |
| Asynchronous matching for scale | Not implemented; matching runs per page request |

## III. METHODOLOGY

SkillLoop follows a modular methodology with seven stages: registration, authentication, skill and profile input, reciprocal matching, recommendation, exchange requests, and collaboration with token incentives. Fig. 1 shows the overall flow of a user through the system, and each stage corresponds to specific views and forms in the code.

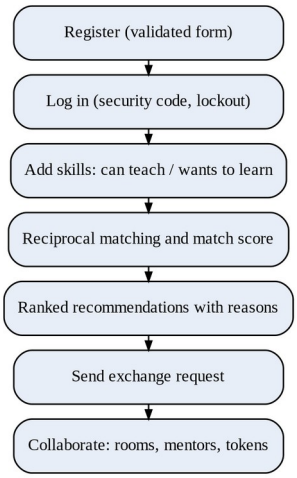



Fig. 1. Flow of a user through SkillLoop.

### A. Registration and Authentication

A visitor provides a username, an e-mail address, a full name and a password with confirmation. The form requires at least six characters and rejects a password mismatch as well as a username or e-mail that already exists, the comparison being case-insensitive. The password is stored using Django's hashing, an empty profile is created, and the user is logged in. Later, the user can log in with either the username or the e-mail address, and the login form requires the security code described in Section V-F.

### B. Skill and Profile Input

On the skills page, a user adds entries, each with a name, a category chosen from ten, a level chosen from four (Beginner to Expert) and a type, either “Can teach” or “Wants to learn”. Only the owner can edit or delete an entry. The profile adds a biography, location, comma-separated interests, an experience level and an optional picture, and a user can tick a box to be listed as a mentor with a short headline. Members can search other members' teach skills by keyword, category and level.

### C. Reciprocal Matching

Let $T_u$ denote the set of lower-cased, whitespace-trimmed names of the skills that user $u$ can teach, and $L_u$ the set of skills $u$ wants to learn. For the current user $u$ and each other active user $v$, the code computes

$$A = T_u \cap L_v,\ \ B = T_v \cap L_u,\ \ n = |A| + |B| \qquad (1)$$

where $A$ contains the skills that $u$ can teach $v$ and $B$ those that $v$ can teach $u$. When $A$ and $B$ are both empty, $v$ is skipped. In every other case the pair counts as reciprocal if both sets are non-empty, and the match score is

$$s = \text{clamp}(\ b + \min(25,\ 8(n - 1)) + \delta,\ 1,\ 100\ ) \qquad (2)$$

with base value $b = 70$ for a reciprocal pair and 35 otherwise, and $\delta = 5$ if the two profiles share at least one interest (compared case-insensitively) and 0 otherwise. The constants are fixed in the code and were not tuned on data. A one-way match can score at most 35 + 25 + 5 = 65, whereas a reciprocal match has $n \geq 2$ and therefore scores at least 70 + 8

= 78. Every reciprocal match thus outranks every one-way match by construction. Results are sorted by $s$ in descending order.

### D. Recommendation Scoring

The recommendation function reuses the match list. For a candidate $v$ with $|S_v|$ skills of either type, it computes

$$r = \text{clamp}( s + \lfloor \min(10, |S_v|) / 2 \rfloor, 1, 100 ) \quad (3)$$

so the activity term contributes between 0 and 5 points, and the list is re-sorted by $r$. For each candidate, the function also builds up to two sentences, naming at most two skills in each direction, that explain why that person was suggested. The interface labels $r$ an "AI Match Score", but the computation involves no learned model: it is a hand-written rule, and comments in the source describe a similarity model as a possible later replacement. The activity term cannot reverse the ordering between reciprocal and one-way matches, because it adds at most 5 to a one-way maximum of 65. Algorithm 1 gives the complete procedure.

**Algorithm 1: Reciprocal matching and recommendation**

**Input:** current user u, all active users V

1. Build $T_u$ and $L_u$ from u's skills.
2. **if** both are empty **then return** empty list.
3. **for each** v in V, v ≠ u **do**
   4. $A \leftarrow T_u \cap L_v$; $B \leftarrow T_v \cap L_u$.
   5. **if** $A$ and $B$ both empty **then continue**.
   6. $b \leftarrow 70$ **if** $A$, $B$ both non-empty **else** 35.
   7. $s \leftarrow b + \min(25, 8(|A|+|B|-1))$.
   8. **if** shared interest **then** $s \leftarrow s + 5$.
   9. Clamp $s$ to [1, 100]; store (v, s, $A$, $B$).
10. Sort matches by $s$; for each, add activity term and reasons, giving $r$.
11. Re-sort by $r$ and return the top entries.

### E. Exchange Requests and Collaboration

From a match or a search result, a user can send a request that names the skill offered and the skill wanted, with an optional message. The receiver may accept or reject the request, and the sender may cancel it. Acceptance records agreement only; the pair then coordinates in a live room or through a mentor booking. A feedback form lets members report problems with a class or with the site, and administrators reply through the Django admin site.

### F. Token Incentive Rules

Each new wallet receives a welcome bonus. Joining a room is charged once per room, so rejoining costs nothing, and a mentor session is charged when it is requested and refunded if the mentor declines or the student cancels while the booking is still pending. A teacher earns a reward only after the named learner confirms the lesson. To limit token farming, the code caps the number of rewarded lessons per teacher in 24 hours, allows one rewarded lesson per learner in that period, and permits at most five lessons awaiting confirmation; a lesson beyond a limit is recorded but earns nothing. Spending is a single guarded update that succeeds only if the balance is sufficient, and every change is written to the ledger together with the resulting balance. The default amounts in the settings file are 50 tokens at sign-up, 1 per room, 2 per mentor request, 3 per confirmed lesson, and at most 3 rewarded lessons per teacher per day.

## IV. System Architecture

The architecture in Fig. 2 follows the three-layer structure of a conventional Django application. The *presentation layer* consists of 26 Django templates, including a shared base layout and a reusable captcha fragment, together with one shared stylesheet. Most pages are rendered on the server and use ordinary form submissions. JavaScript appears only where a page must update without reloading: the live-room chat polls the server, the video call exchanges signalling messages, the assistant widget posts questions, and the captcha image can be refreshed.

The *application layer* is a single Django app named swap, in which 42 URL routes map to view functions. Views hand validation to forms, scoring to a matching module, wallet rules to a token module, the captcha and login throttling to security modules, and chatbot replies to an assistant module. Authentication, sessions, CSRF protection and password hashing come from Django's built-in components. The *data layer* uses Django's object-relational mapper on top of an SQLite file. There are two optional external dependencies: browsers contact public STUN servers, and optionally a TURN relay, to set up peer-to-peer media connections, and the assistant can call an external language-model API if a key is present in the environment.

The data model has 13 application models in addition to Django's User model (Fig. 3). They fall into five groups: identity and skills (UserProfile, Skill), exchange (SkillRequest, MentorBooking), collaboration (SkillRoom, RoomMessage, CallPresence, CallSignal), incentives (TokenWallet, TokenTransaction, TeachSession) and support (Feedback, ChatMessage).

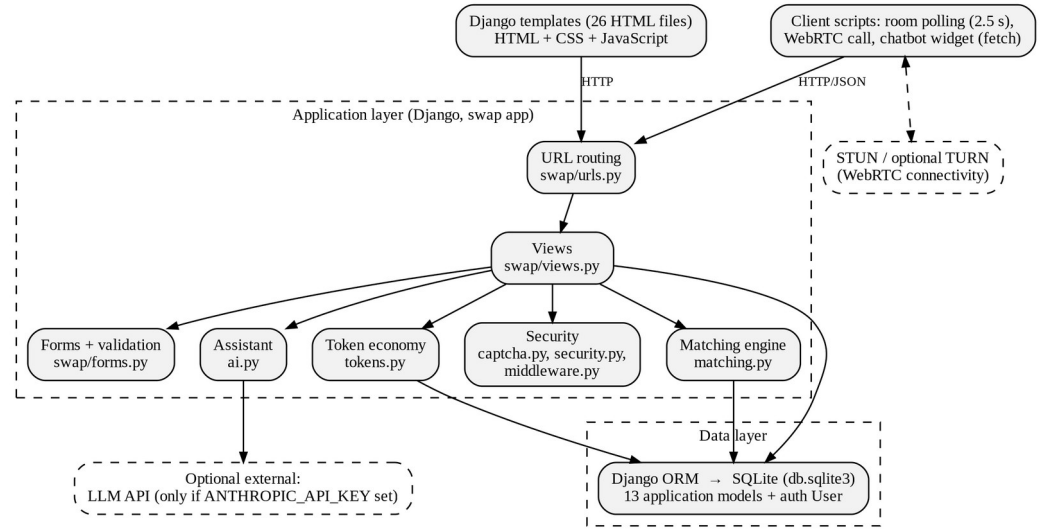



Fig. 2. System architecture of SkillLoop derived from the source code. Dashed boxes are optional or external services.

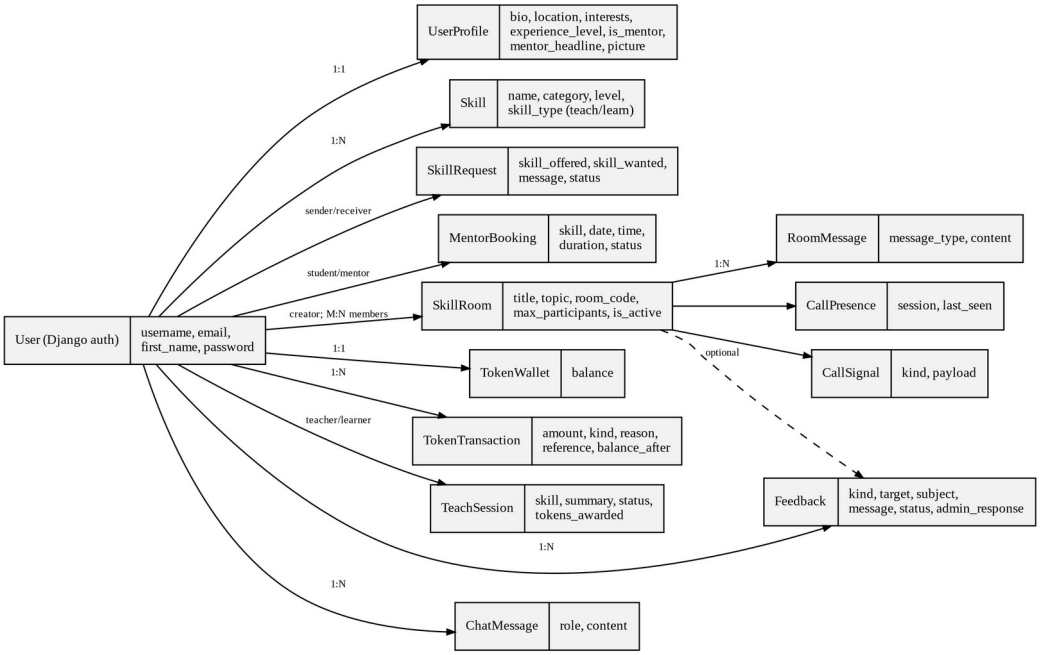



Fig. 3. Data model of SkillLoop: 13 application models and their relationships to Django's User model (attributes abbreviated).

## V. Implementation

### A. Technology Stack

Table II summarizes the technologies identified in the project files. The application is written in Python with Django, data is stored in SQLite, and the browser side uses plain JavaScript and the WebRTC interface. Running the core features requires no separate machine-learning infrastructure or paid external service.

TABLE II
Technology Stack of SkillLoop

| Layer | Technology |
|---|---|
| Language / framework | Python; Django (requirements.txt: ≥ 5.0 and < 5.3) |
| Database | SQLite through the Django ORM |
| Imaging | Pillow (captcha image, profile pictures) |
| Front end | Django templates, HTML, one CSS file, JavaScript (fetch, WebRTC) |
| Live updates | HTTP polling for chat and call signalling; no WebSockets |
| Video / voice | Browser WebRTC, peer-to-peer mesh, STUN, optional TURN [13] |
| Assistant | Keyword rule base; optional external LLM call if a key is set |
| Cache | Django local-memory cache (captcha records, lockouts) |
| Client requirement | Modern browser; camera and microphone for video |

### B. Skills, Matching and Recommendation Modules

Members manage their “teach” and “learn” lists on the skills page (Fig. 4(a)). The dashboard and the matches page call the matching function and display each candidate with a score and the skills that overlap in both directions (Fig. 4(b)). The recommendations page lists the top twelve suggestions with the recommendation score and the generated reasons (Fig. 4(c)), while the dashboard shows the top three.

### C. Exchange, Rooms and Video Calls

The exchange page lists incoming and outgoing requests with accept, reject and cancel actions (Fig. 4(d)). A live room has a title, an optional topic, an eight-character code generated from a random UUID, and a capacity chosen from 2, 3, 5, 10, 25 and 50. Joining requires the security code, and the capacity check, token charge and membership insert run inside one database transaction guarded by a row lock and a process-level lock, so simultaneous joins cannot exceed the capacity. Participants post messages typed as message, question, answer or note. The chat client asks for newer messages every 2.5 seconds, passing the identifier of the last message it has seen, and the server returns up to 100 newer messages along with the participant list.

For video, up to six participants can be in the call at once (Fig. 4(e)); other room members can still use the chat. The server stores only presence records, refreshed by a heartbeat and treated as stale after 12 seconds, and short handshake messages of type offer, answer or candidate. Media flows directly between browsers, in line with the WebRTC architecture [13]. The signalling endpoint rejects unknown message types, payloads above 20,000 characters, senders who are not currently in the call, and recipients whose session does not match.

### D. Token Wallet and Mentors

The tokens page shows the balance, the ledger and the lessons awaiting confirmation (Fig. 4(f)). A member listed as a mentor can be booked for a date, time and duration; the mentor can confirm or decline, and either side can cancel while the booking is pending, following the refund rules of Section III-F.

### E. Database Design

Skills are kept in a single table with a type field instead of two tables, so the same search and matching code can treat “teach” and “learn” entries symmetrically, and the append-only token ledger makes every balance change auditable.

### F. Validation and Security

All application views except the home page, registration, login and captcha endpoints require a logged-in user, and objects are fetched with ownership constraints; for example, a skill can be edited only by its owner and a request can be answered only by its receiver. Django's CSRF middleware is enabled, the session cookie is HttpOnly with SameSite set to Lax, and the standard password validators are configured. The security code is a self-hosted image challenge. A signed token carries a random nonce and a timestamp, the expected code is derived from the nonce with a keyed hash (HMAC-SHA256), the token expires after 15 minutes, and a code can be tried only once because used nonces are recorded in the cache. Forms with this check also carry a hidden honeypot field and reject submissions made within two seconds of rendering. The check is required at login and when joining a room. After five failed logins for the same address and identifier, further attempts are refused for ten minutes.

### G. Assistant

The assistant scores each entry of a small knowledge base by counting how many of its keywords appear in the question, and answers with the best-scoring entry or a fallback reply. If an API key is set in the environment, the code first tries an external language-model service and silently falls back to the rule base on any error. Conversations are stored per user. The external path was not exercised for this paper.




*Fig. 4. SkillLoop implementation screenshots: (a) dashboard, (b) live room with chat and video, (c) mentor directory, (d) mentor booking form, (e) bookings, and (f) user profile and skills. Screenshots are taken from the project presentation.*

## VI. Results and Discussion

No controlled experiment or user study has been conducted on SkillLoop, so no accuracy, response-time, satisfaction or scalability figures are reported, and none should be inferred. The available evidence consists of the worked scoring examples, the inventory of verified modules, the automated test suite and the interface screenshots of Fig. 4.

### A. Evaluation of the Matching Module

Table III gives six scenarios calculated by hand from Eqs. (2) and (3). They illustrate how the rule behaves and are not measurements from user data. Scenario S1 corresponds to the example in the project abstract, where one student teaches Python and wants to learn Artificial Intelligence and another has the opposite profile, with no shared interest. The scenarios show that the overlap bonus grows with the number of shared skills until it saturates (S3 to S6), that a shared interest adds five points (S1 to S2), and that every reciprocal scenario scores above every one-way scenario.

TABLE III
WORKED SCORING EXAMPLES FROM EQS. (2) AND (3)

| Scenario (candidate) | $\lvert A \rvert + \lvert B \rvert$ | Shared interest | Match s | Rec. r |
|---|---|---|---|---|
| S1: reciprocal 1 + 1; 4 skills | 2 | No | 78 | 80 |
| S2: as S1 with shared interest | 2 | Yes | 83 | 85 |
| S3: reciprocal 2 + 2; 6 skills | 4 | No | 94 | 97 |
| S4: one-way, 1 overlap; 3 skills | 1 | No | 35 | 36 |
| S5: one-way, 2 overlaps; 3 skills | 2 | Yes | 48 | 49 |
| S6: reciprocal 5 + 5; 12 skills | 10 | Yes | 100 | 100 |

### B. Implemented Functionality

The modules were checked against the source code. Registration, skills, matching, recommendation, requests, rooms, video calls, mentor booking, tokens and the security mechanisms are all present, while project recommendation is not (see also Table I).

### C. Automated Tests

The test file tests.py defines 23 test methods in six classes. They cover the welcome bonus, the captcha on the login, registration, feedback and room-join pages, WebRTC signal exchange and call capacity, the room workflow (capacity, inactive rooms, a single token charge, participant-only chat), the hardening of the signalling endpoints, and the presence of a STUN server. Automated tests do not exist for the matching and recommendation functions, the request workflow, mentor booking, the reward rules or the assistant. Running “python manage.py test swap” could not be verified in this environment, so no pass/fail count is claimed.

### D. Discussion

Three points follow from the implementation. First, the ranking is deterministic and explainable: any score can be reproduced from two skill lists and two profiles, and the ordering of reciprocal above one-way matches holds by construction, so the behaviour is easy to inspect and test. Second, a match is only the beginning of an exchange. Rooms, mentors and a reward for teaching give matched users a way to turn a suggestion into actual learning activity, an aspect the base work [1] leaves unspecified. Third, a rule-based score is a reasonable starting point, but its quality depends on the skill names users enter, which is why the limitations below matter.

### E. Privacy and Ethical Considerations

The platform stores personal profile data, hashed passwords, chat messages and mentor bookings, so deployments should obtain consent, restrict access, and review the assistant's optional external call before enabling it, since user questions would then leave the server. Video media travels peer to peer and is not stored by the application. Because tokens give access to sessions, the daily limits and learner confirmation should be monitored for abuse.

### F. Limitations

Skills are compared as exact lower-cased strings, so “UI/UX Design” and “ui ux design”, or “ML” and “machine learning”, do not match; category and level are stored but are not used in scoring. The score constants are not calibrated, and no experiment shows that higher scores lead to successful exchanges. Matching runs two queries per candidate on every page that uses it, so its cost grows linearly with the number of users. The configuration is a development setup (debug mode on, all hosts allowed, secret key in the settings file, process-local cache), chat relies on polling, and the video call is a full mesh limited to six participants. The request status “Completed” is defined but no view sets it, and the assistant's rule base is small.

## VII. CONCLUSION AND FUTURE WORK

This paper presented SkillLoop, a browser-based platform that turns the idea of reciprocal skill exchange into a working application. Users declare what they can teach and what they want to learn, a transparent rule computes a reciprocal score, and a recommendation layer explains each suggestion. Requests, live rooms, mentor bookings and a teaching-based token mechanism let matched users begin learning together, while a captcha, honeypot and login throttling protect the entry points. Compared with the base work, the implementation realises profile-based reciprocal matching and collaboration, and leaves embeddings, reputation ranking, notifications and project recommendation open. The claims are limited to implemented features, and the effect of the platform on learning has not been measured.

Future work includes the project-recommendation section proposed in [1], normalisation of skill names, use of category and level in the score, and semantic similarity such as embeddings, which would also make the “AI” label accurate. Further extensions are matching among three or more users, a reputation signal based on confirmed lessons, and match notifications. On the engineering side, production settings, a shared cache, a server database, WebSocket updates and automated tests for the matching module are needed. A formal evaluation with real users should measure, for example, the share of suggestions that lead to accepted requests and completed sessions, and compare the score against a keyword baseline.

## REFERENCES


[1] S. Raj Kumar, R. Jai Shree Ram, and S. Rajagopal, “Skill exchange platform with intelligent skill matching,” *IRE J.*, vol. 9, no. 10, pp. 212–215, Apr. 2026, doi: 10.64388/IREV9I10-1715909.

[2] K. J. Topping, “Trends in peer learning,” *Educ. Psychol.*, vol. 25, no. 6, pp. 631–645, 2005.

[3] P. Dillenbourg, “What do you mean by collaborative learning?,” in *Collaborative-Learning: Cognitive and Computational Approaches*, P. Dillenbourg, Ed. Oxford, U.K.: Elsevier, 1999, pp. 1–19.

[4] R. D. Roscoe and M. T. H. Chi, “Understanding tutor learning: Knowledge-building and knowledge-telling in peer tutors’ explanations and questions,” *Rev. Educ. Res.*, vol. 77, no. 4, pp. 534–574, 2007.

[5] L. Fiorella and R. E. Mayer, “The relative benefits of learning by teaching and teaching expectancy,” *Contemp. Educ. Psychol.*, vol. 38, no. 4, pp. 281–288, 2013, doi: 10.1016/j.cedpsych.2013.06.001.

[6] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.

[7] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.

[8] N. Manouselis, H. Drachsler, R. Vuorikari, H. Hummel, and R. Koper, “Recommender systems in technology enhanced learning,” in *Recommender Systems Handbook*, Boston, MA, USA: Springer, 2011, pp. 387–415.

[9] L. Pizzato, T. Rej, T. Chung, I. Koprinska, and J. Kay, “RECON: A reciprocal recommender for online dating,” in *Proc. ACM Conf. Recommender Syst. (RecSys)*, 2010, pp. 207–214, doi: 10.1145/1864708.1864747.

[10] I. Palomares, C. Porcel, L. Pizzato, I. Guy, and E. Herrera-Viedma, “Reciprocal recommender systems: Analysis of state-of-art literature, challenges and opportunities towards social recommendation,” *Inf. Fusion*, vol. 69, pp. 103–127, 2021.

[11] C. E. Hmelo-Silver, “Problem-based learning: What and how do students learn?,” *Educ. Psychol. Rev.*, vol. 16, no. 3, pp. 235–266, 2004.

[12] J. W. Thomas, “A review of research on project-based learning,” The Autodesk Foundation, San Rafael, CA, USA, 2000.

[13] H. Alvestrand, “Overview: Real-time protocols for browser-based applications,” IETF, RFC 8825, Jan. 2021, doi: 10.17487/RFC8825.

# SkillLoop: A Reciprocal Skill-Exchange Platform with Rule-Based Match Scoring, Token Incentives, and Live Collaboration Rooms

Veerajebika.R[1], S.Pooja[2], Gracy Rose.A[3] ,Dr.G.Revathy [4]
[4]Associate Professor,[1,2,3]UG Scholar
Department of Artificial Intelligence and Machine Learning, Vels Institute of Technology Pallavaram ,India
Email id: veerajebika@gmail.com, poojasenthil052007@gmail.com, zoegracyrose@gmail.com , revathy.se@vistas.ac.in

## Abstract

Technical skills are often learned outside the classroom, where many learners lack access to an instructor or a paid mentor. Peer learning offers an alternative, but existing platforms rarely model the two-sided condition that keeps an exchange sustainable, namely that each participant can teach what the other wishes to learn. This paper presents SkillLoop, a web-based platform for reciprocal peer-to-peer skill exchange. Users declare the skills they can teach and the skills they want to learn, and the system pairs them with members whose profiles complement their own. Matching compares skill sets in both directions and assigns an interpretable rule-based score from 1 to 100, constructed so that every reciprocal pair ranks above every one-way pair. A recommendation layer adds an activity term and generates a textual reason for each suggestion. Matched users collaborate through exchange requests, live rooms with text chat and browser-based video calls (WebRTC), and mentor bookings. A token wallet with an append-only ledger rewards confirmed teaching within anti-abuse limits. The platform is implemented in Python with Django and SQLite, and it includes a self-hosted captcha, a honeypot field, and login throttling. The evidence reported consists of worked scoring examples, a verified module inventory, and an automated test suite covering security and room workflows. Skills are currently matched as exact strings, the score constants are uncalibrated, and no user study has yet been conducted. Planned work includes semantic skill matching, project recommendation, and a formal evaluation with real users.

**Keywords—** Skill Exchange, Peer Learning, Reciprocal Matching, Recommender System, Learning by Teaching, Django, WebRTC

# SkillLoop: A Reciprocal Skill-Exchange Platform with Rule-Based Match Scoring, Token Incentives, and Live Collaboration Rooms

Veerajebika.R, S.Pooja, Gracy Rose.A,Dr.G.Revathy
Department of Artificial Intelligence and Machine Learning, Vels Institute of Technology Pallavaram ,India

**Abstract**

Technical skills are often learned outside the classroom, where many learners lack access to an instructor or a paid mentor. Peer learning offers an alternative, but existing platforms rarely model the two-sided condition that keeps an exchange sustainable, namely that each participant can teach what the other wishes to learn. This paper presents SkillLoop, a web-based platform for reciprocal peer-to-peer skill exchange. Users declare the skills they can teach and the skills they want to learn, and the system pairs them with members whose profiles complement their own. Matching compares skill sets in both directions and assigns an interpretable rule-based score from 1 to 100, constructed so that every reciprocal pair ranks above every one-way pair. A recommendation layer adds an activity term and generates a textual reason for each suggestion. Matched users collaborate through exchange requests, live rooms with text chat and browser-based video calls (WebRTC), and mentor bookings. A token wallet with an append-only ledger rewards confirmed teaching within anti-abuse limits. The platform is implemented in Python with Django and SQLite, and it includes a self-hosted captcha, a honeypot field, and login throttling. The evidence reported consists of worked scoring examples, a verified module inventory, and an automated test suite covering security and room workflows. Skills are currently matched as exact strings, the score constants are uncalibrated, and no user study has yet been conducted. Planned work includes semantic skill matching, project recommendation, and a formal evaluation with real users.

**Keywords—** Skill Exchange, Peer Learning, Reciprocal Matching, Recommender System, Learning by Teaching, Django, WebRTC

Institute of Computer Science and Engineering, SIMATS Engineering, Chennai.
ISBN No.: 978-93-344-7370-4

# SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM | NAAC A++ GRADE | nirf 45 RANK

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

VEERAJEBIKA. R

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

A Reciprocal Skill Exchange and AI - Assisted

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

N. B

Conference Chair
ICAI-CME 2026

B. Ramesh

Principal
**Dr. B. Ramesh**

# SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

POOJA.S

Vels Institute of Science and Technology & Advanced Studies

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

A Reciprocal Skill Exchange and AI - Assisted.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

ENGINEER TO EXCEL

# SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

QS I-GAUGE INDIAN UNIVERSITY RATING PLATINUM | NAAC A++ GRADE | nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL | MoE's INNOVATION CELL (GOVERNMENT OF INDIA) | IQAC | SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

GRACY ROSE. A

Vels Institute of Science and Technology & Advanced Studies.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

A Reciprocal Skill Exchange and AI-Assisted.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

N. Bhl

Conference Chair
ICAI-CME 2026

B. Ramesh

Principal
**Dr. B. Ramesh**

ENGINEER TO EXCEL

# SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

QS I·GAUGE INDIAN UNIVERSITY RATING PLATINUM

Accredited by NAAC A++ GRADE

nirf 45 RANK

INSTITUTION'S INNOVATION COUNCIL (Ministry of Education Initiative)

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

IQAC

SUSTAINABLE DEVELOPMENT GOALS

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

**Dr. G. Revathy**

Vels Institute of Science and Technology Advances Studies

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

A reciprocal skill exchange and AI assisted peer leaning platform

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Principal
**Dr. B. Ramesh**